

Fundamentos de la programación

Ejercicios. Hoja 2

Desarrollar los siguientes ejercicios utilizando expresiones numéricas y booleanas, así como asignación (**NO** pueden utilizarse condicionales *if-else*, ni tampoco bucles).

Los ejercicios marcados como **Evaluable** tienen que entregarse a través de CV durante la sesión de laboratorio, previa corrección por parte del profesor.

- Se subirá **únicamente** el archivo **Program.cs**
- Sólo hará la entrega uno de los miembros del grupo.
- La línea 1 (y siguientes) deben contener los nombres de los alumnos que hacen la entrega de la forma:

```
// Nombre Apellido1 Apellido2
```

1. Escribe un programa que solicite un tiempo en horas, minutos y segundos, y devuelva ese tiempo en segundos. Declara las variables necesarias con los tipos adecuados.
2. Escribe un programa que solicite las edades de tres alumnos y calcule la edad media de los mismos. Declara las variables necesarias con los tipos adecuados.
3. Escribe un programa que calcule la nota final de la asignatura de FP, a partir de las calificaciones obtenidas en el examen final, las 2 prácticas y la actividad adicional. El programa debe pedir al usuario cada uno de estos datos y mostrará en pantalla la nota final obtenida. No tendrá en cuenta los requisitos de mínimos del examen final (al menos una calificación de 5), ni de las prácticas (todas aprobadas).
4. Escribir un programa que solicite un número de 4 cifras, lo descomponga en dígitos (con división y módulo) y determine las condiciones siguientes (debe escribir en pantalla el resultado de cada una de ellas):
 - alguna de las cifras es un 3.
 - al menos dos de sus cifras son 3.
 - exactamente dos de sus cifras son 3 y además son consecutivas.
 - si es capicúa (se lee igual de izquierda a derecha y de derecha a izquierda).
5. Escribir un programa que solicite un número binario de 4 cifras y calcule su valor en decimal. Por ejemplo, para el número 1001 debe calcular el valor 9.
6. (**Evaluable**) Queremos diseñar un programa para un cajero automático, que calcule la cantidad de billetes y monedas que hay que darle al cliente cuando retira un determinado importe del banco. La cantidad de dinero se solicitará en dos variables enteras, euros y céntimos. En primer lugar, el programa calculará el total en céntimos. A partir de ese valor de se calculará el número de billetes y monedas de distintos valores que tiene que utilizar para sumar esa cantidad, *maximizando el número de billetes/monedas de mayor a menor valor*. Es decir, primero calculará el máximo número de billetes de 500 euros; después, para la cantidad restante, el máximo número de billetes de 200 euros; luego el máximo de billetes de 100 euros, y así sucesivamente hasta llegar a las monedas de céntimo. **Todas las variables declaradas deben ser de tipo entero**. Una posible ejecución puede ser:

Dinero a cambiar: 1494,69

Tu cambio:

Billetes de 500:	2	Monedas de 2:	2
Billetes de 200:	2	Monedas de 1:	0
Billetes de 100:	0	Monedas de 50c:	1
Billetes de 50:	1	Monedas de 20c:	0
Billetes de 20:	2	Monedas de 10c:	1
Billetes de 10:	0	Monedas de 5c:	1
Billetes de 5:	0	Monedas de 2c:	2
		Monedas de 1c:	0

7. Se dice que un año es *bisiesto* si es divisible entre 4, a menos que sea divisible entre 100. Sin embargo, si un año es divisible entre 100 y además es divisible entre 400, también resulta bisiesto. Escribir un programa que pida un año al usuario y determine si es o no bisiesto.
8. (**Evaluable**) Implementar un programa que solicite 3 enteros positivos correspondientes a los lados de un triángulo y decida si definen:
- Un triángulo
 - Un triángulo equilátero
 - Un triángulo isósceles
 - Un triángulo escaleno.
 - Un triángulo rectángulo.

Un posible ejemplo de ejecución:

```
lado 1: 3
lado 2: 6
lado 3: 6
Triangulo: True
Equilatero: False
Isosceles: True
Escaleno: False
Rectangulo: False
```

9. Realizar el mismo ejercicio utilizando reales en vez de enteros, teniendo en cuenta los errores derivados de la representación en coma flotante. Para ello puedes definir la constante `EPSILON = 1e-10` y considerar que dos reales son iguales si su diferencia en valor absoluto es menor que esta constante.
10. Escribe un programa para hacer el cálculo de un préstamo hipotecario tomando como datos el *capital* prestado, el *interés* anual y el *plazo* (la duración en años). El programa mostrará la *cuota* mensual a pagar, el *total* pagado una vez terminado el plazo y la cantidad correspondiente a los *intereses* una vez finalizado el préstamos. La fórmula que nos da la cuota mensual es:

$$cuota = \frac{capital \times interes}{100 \times (1 - (1 + \frac{interes}{100})^{-plazo})}$$

Donde *capital* es el capital prestado, *interes* corresponde al *interés mensual* (el anual dividido entre 12) y *plazo* al *plazo en meses* (en la entrada se expresa en años), i.e., hay que hacer conversiones de unidades. Para calcular la potencia $(1 + \frac{interes}{100})^{-plazo}$ se puede buscar el método correspondiente en la guía de referencia de C# de Microsoft. Un ejemplo de ejecución sería el siguiente:

```
Capital: 140000
Interés: 6
Plazo: 20

Resultado:
Cuota: 1003,00348186944 euros
Total: 240720,835648667 euros
Intereses: 100720,835648667 euros
```

Puedes hacer más cálculos y comparar los resultados con los de <http://www.calculohipoteca.org/>.

En los siguientes ejercicios utiliza el compilador para experimentar y confirmar tus respuestas.

11. Suponiendo que `n` es de tipo `int`, empareja las descripciones con la definición correspondiente:
- El mayor número par no superior a `n`
 - El primer número par mayor o igual que `n`
 - El primer impar mayor o igual que `n`
 - $(n / 2) * 2$
 - $n + ((n+1) \% 2)$
 - $((n+1)/2) * 2$
12. Determinar cuáles de las siguiente expresiones son erróneas y cuáles correctas. En el caso de las expresiones correctas indicar el tipo que deben tener las variables que aparecen, el tipo de la expresión completa y el valor resultante de la evaluación, si es posible calcularlo:
- `x == y`
 - `p > p+1`
 - `p == true`
 - `(k % 2 != 0) || ((k+1) % 2 != 0)`
 - `(2*2)/4 == 2*(2/4)`
 - `'b'-'a' > 0`
 - $((n*(n-1)) \% 2 != 0)$
 - `p == q || r`
 - `10/3 == 10/3.0`

13. Dadas las variables booleanas `x`, `y`, `z`, con valores `x=true`, `y=false`, `z=true`, evaluar las expresiones que aparecen a continuación:

- | | |
|---|---------------------------------------|
| ■ <code>(x && y) (x && z)</code> | ■ <code>x y && z</code> |
| ■ <code>(x !y) && (!x z)</code> | ■ <code>!(x y) && z</code> |

14. Indicar cuáles de las siguientes asignaciones son correctas y determinar el tipo que deben tener las variables que aparecen:

- | | |
|-------------------------------|--|
| ■ <code>a = (b==c==d);</code> | ■ <code>a = 2.3 / b;</code> |
| ■ <code>x = 2<1;</code> | ■ <code>2 = a;</code> |
| ■ <code>x = x+1;</code> | ■ <code>p = Math.Sqrt(2) == Math.Sqrt(2.0);</code> |
| ■ <code>c = 2 <= 7;</code> | ■ <code>x = 'y';</code> |

15. Las siguientes expresiones tienen sentido, pero no son válidas en la sintaxis de C#. Traducirlas a expresiones booleanas válidas.

`x < y <= z` `a > b > c` `x` es diferente de `a` y de `b`
Dos cualesquiera de los valores `x`, `y`, `z` son iguales